Strojenie poleceń SQL – zadania

Wstęp

1. Przyłącz się do swojej bazy danych jako użytkownik SYS. Sprawdź, czy jest jeszcze wolne miejsce w przestrzeni tabel USERS. Jeśli miejsce jest bliskie wyczerpania, sprawdź, czy plik implementujący przestrzeń tabel USERS ma aktywną własność automatycznego zwiększania rozmiaru. Jeśli ta własność jest nieaktywna, włącz ją. Odczytaj nazwę tymczasowej przestrzeni tabel w Twojej bazie danych, a następnie sprawdź, czy plik implementujący tymczasową przestrzeń tabel ma aktywną własność automatycznego zwiększania rozmiaru. Jeśli własność jest nieaktywna, aktywuj ją.

```
select bytes/1024 from dba_free_space where tablespace_name = 'USERS';

select file_name, autoextensible
from dba_data_files
where tablespace_name = 'USERS';

alter database datafile '/home1/admXX/oradata/users01.dbf'
autoextend on next 5M maxsize 50M;

select tablespace_name
from dba_tablespaces
where contents = 'TEMPORARY';

select file_name, autoextensible
from dba_temp_files join dba_tablespaces using (tablespace_name)
where contents = 'TEMPORARY';

alter database tempfile '/home1/admXX/oradata/TEMP1.dbf' autoextend on
next 5M maxsize 50M;
```

2. Utwórz konto użytkownika OPT_USER, w schemacie którego będziesz wykonywał/a ćwiczenia. Domyślną przestrzenią tabel użytkownika powinna być przestrzeń USERS. Nadaj użytkownikowi prawa przyłączania się do bazy danych i tworzenia obiektów.

```
create user opt_user identified by opt_user
default tablespace users quota unlimited on users;

grant create session, resource to opt_user;
```

3. Utwórz rolę PLUSTRACE wykonując skrypt `plustrce.sql`, zlokalizowany w katalogu `$ORACLE_HOME/sqlplus/admin`.

4. Nadaj użytkownikowi OPT_USER rolę PLUSTRACE.

```
grant plustrace to opt_user;
```

5. Przyłącz się do bazy danych jako użytkownik OPT_USER, utwórz potrzebne do ćwiczeń tabele i wypełnij je danymi. Użyj do tego celu skryptu **`/ora1/oracle/opt.sql`**. Sprawdź strukturę i liczbę rekordów utworzonych przez skrypt relacji PRAC i ZESP.

6. Wyjaśnij plan poniższego zapytania, używając polecenia EXPLAIN_PLAN, nadaj wygenerowanemu planowi identyfikator „p1”:

```
select nazwa, count(*) from zesp natural join prac
group  by nazwa
order  by count(*) desc;
```

Następnie odczytaj plan zapytania, używając:

- zapytania:
  ```
  select operation,object_name,id,cost,parent_id
  from plan_table where statement_id='p1' order by id;
  ```
- skryptu: **$ORACLE_HOME/rdbms/admin/utlxpls.sql**

```
explain plan
set statement_id = 'p1'
for select nazwa, count(*) from zesp natural join prac
group  by nazwa
order  by count(*) desc;
```

7. Włącz dyrektywę `SET AUTOTRACE ON EXPLAIN`. Sprawdź ponownie plan wykonania zapytania z punktu 6., wykorzystując działanie zastosowanych dyrektyw.

Uwaga! W następnych zadaniach do wyjaśniania planu wykonania zapytania używamy dyrektywy AUTOTRACE.

Metody dostępu

8. Wyjaśnij plan poniższego zapytania. Jaką metodą dostępu wybrał optymalizator do wykonania tego zapytania?

```
select rowid, nazwisko, plec, placa
from   prac
where  id_prac = 900;
```

9. Zdefiniuj zapytanie, które odczyta te same dane, co zapytanie z zadania 8., posłuż się w tym celu adresem rekordu (ROWID), odczytanym w zadaniu 8. Następnie wyjaśnij plan tego zapytania. Jaka metoda dostępu do danych została użyta?

```
select rowid, nazwisko, plec, placa
from   prac
where  rowid = 'AAABbYAADAAAAA0AAN';
```

10. Wyłącz dyrektywę `AUTOTRACE`. Sprawdź w słowniku bazy danych, jakie indeksy założono na relacji PRAC.

```
select index_name
from user_indexes
where table_name = 'PRAC';
```

11. Utwórz indeks typu B-drzewo o nazwie PRAC_IDX na atrybucie ID_PRAC relacji PRAC. Następnie ponownie wyjaśnij plan zapytania z punktu 8. Czym różni się otrzymany plan?

```
create index prac_idx on prac(id_prac);
```

12. Usuń indeks PRAC_IDX. Następnie na atrybucie ID_PRAC relacji PRAC zdefiniuj klucz podstawowy o nazwie PRAC_PK.

```
drop index prac_idx;
alter table prac add constraint prac_pk primary key(id_prac);
```

13. Ponownie sprawdź w słowniku bazy danych indeksy dla relacji PRAC. Wyświetl dodatkowo nazwy poindeksowanych atrybutów. Co zauważyłeś/aś?

```
select c1.index_name, uniqueness, column_name, column_position
from   user_indexes c1, user_ind_columns c2
where  c1.table_name = 'PRAC'
and    c1.index_name = c2.index_name;
```

14. Ponownie wyjaśnij plan zapytania z punktu 8. Czym różni się otrzymany plan?

15. Utwórz indeks typu B-drzewo o nazwie PRAC_NAZWISKO_IDX na atrybucie NAZWISKO relacji PRAC. Następnie wykonaj poniższe zapytania, dla każdego wyjaśniając jego plan.

```
create index prac_nazwisko_idx on prac(nazwisko);

select * from prac where nazwisko = 'Prac155';
select * from prac where nazwisko like 'Prac155%';
select * from prac where nazwisko like '%Prac155%';
select * from prac where nazwisko like 'Prac155%'
                     or nazwisko like 'Prac255%';
```

Dlaczego w niektórych zapytaniach optymalizator nie użył indeksu?

16. Usuń indeks PRAC_NAZWISKO_IDX i na jego miejsce utwórz skonkatenowany indeks typu B-drzewo o nazwie PRAC_NAZW_PLACA_IDX na atrybutach NAZWISKO i PLACA relacji PRAC. Następnie wykonaj poniższe zapytania, dla każdego wyjaśniając jego plan.

```
drop index prac_nazwisko_idx;
create index prac_nazw_placa_idx on prac(nazwisko, placa);

select count(*) from prac where nazwisko like 'Prac1%';
select count(*) from prac where nazwisko like 'Prac1%' and placa > 100;
select count(*) from prac where placa > 100;
```

Jakich operacji optymalizator użył przy wykonaniu powyższych zapytań?

17. Utwórz indeks typu B-drzewo o nazwie PRAC_PLEC_IDX na atrybucie PLEC relacji PRAC. Następnie wykonaj poniższe zapytanie.

```
create index prac_plec_idx on prac(plec);

select count(*) from prac where plec='M'
and id_prac between 100 and 110;
```

Czy optymalizator użył utworzonego indeksu?

18. Wyłącz dyrektywę AUTOTRACE. Następnie sprawdź, czy dla relacji PRAC zebrano statystyki. Skorzystaj z poniższych zapytań.

```
select table_name, last_analyzed, num_rows from user_tables
where table_name='PRAC';

select column_name, num_distinct, low_value, high_value
       num_buckets from user_tab_columns where table_name = 'PRAC';

select index_name, last_analyzed, num_rows from user_indexes where
table_name='PRAC';

select * from user_tab_histograms where table_name='PRAC';
```

19. Zbierz statystyki dla relacji PRAC. Następnie ponownie wykonaj zapytania z 18. Jakie rodzaje statystyk polecenie zgromadziło?

```
exec dbms_stats.gather_table_stats(ownname=>'OPT_USER',tabname=>'PRAC');
```

20. Wykonaj poniższe zapytanie. Co zauważyłeś/aś? Czy przy odpowiedzi na poniższe zapytanie optymalizator korzysta z relacji PRAC?

```
select count(*) from prac;
```

21. Utwórz indeks typu B-drzewo o nazwie PRAC_CZY_ETAT_IDX na atrybucie CZY_ETAT relacji PRAC. Zbierz statystyki dla tego indeksu. Następnie wykonaj poniższe zapytanie i zanalizuj jego plan.

```
create index prac_czy_etat_idx on prac(czy_etat) compute statistics;

select count(*) from prac where czy_etat='T' and plec='K';
```

22. Usuń indeksy na atrybutach PLEC i CZY_ETAT, wykonaj ponownie zapytanie z zadania 21. Czy widzisz różnice w obu planach?

```
drop index prac_plec_idx;
drop index prac_czy_etat_idx;
```

23. Utwórz indeksy bitmapowe na relacji PRAC, na atrybutach PLEC i CZY_ETAT, o nazwach odpowiednio PRAC_PLEC_BMP_IDX i PRAC_CZY_ETAT_BMP_IDX. Zbierz statystyki dla obu indeksów.

```
create bitmap index prac_plec_bmp_idx on prac(plec) compute statistics;
create bitmap index prac_czy_etat_bmp_idx on prac(czy_etat) compute
statistics;
```

24. Wykonaj poniższe zapytanie:

```
select count(*) from prac where czy_etat='T' and plec='K';
```

Czy przy odpowiedzi na powyższe zapytanie optymalizator użył indeksów?

25. Utwórz indeks typu B-drzewo o nazwie PRAC_PLACA_IDX na atrybucie PLACA relacji PRAC. Zbierz statystyki dla tego indeksu.

```
create index prac_placa_idx on prac(placa) compute statistics;
```

26. Sprawdź plany następujących zapytań:

```
select nazwisko from prac where placa < 2;
select nazwisko from prac where ROUND(placa) < 2;
```

Dlaczego drugie zapytanie nie korzysta z indeksu na płacy?

27. Utwórz indeks funkcyjny o nazwie PRAC_PLACA_FUN_IDX na relacji PRAC, który będzie używany przy zapytaniu o zaokrągloną wartość płacy pracownika. Wykonaj ponownie zadanie 26. Jakie różnice zaobserwowałeś/aś?

```
create index prac_placa_fun_idx on prac(round(placa));
```

Sortowanie

28. Porównaj plany wykonania następujących zapytań (pamiętaj, by przed wykonaniem zapytań wyłączyć wypisywanie wyników).

```
set autotrace traceonly;

select * from prac order by id_prac;
select * from prac order by id_prac desc;
select * from prac order by nazwisko;
select distinct nazwisko from prac;
select nazwisko from prac group by nazwisko;
```

Tabele IOT

29. Zbuduj tabelę o organizacji indeksowej o nazwie PRAC_ZESP_IOT i zapełnij danymi z tabel prac i zesp, wykonując poniższe zapytanie.

```
CREATE TABLE prac_zesp_iot (
id_prac NUMBER PRIMARY KEY,
nazwisko VARCHAR2(20),
nazwa VARCHAR2(20) )
ORGANIZATION INDEX
PCTTHRESHOLD 20
OVERFLOW TABLESPACE USERS;

insert into prac_zesp_iot select id_prac, nazwisko, nazwa
from prac, zesp where prac.id_zesp = zesp.id_zesp;
```

Sprawdź plan wykonania poniższego zapytania do takiej tabeli.

```
select * from prac_zesp_iot where id_prac < 100;
```

Połączenia

30. Ustaw tryb wyjaśniania zapytań dyrektywą
    `set autotrace traceonly explain`

31. Sprawdź plan wykonania następujących zapytań z połączeniem naturalnym:

```
select count(*) from zesp natural join prac;
select count(*) from prac natural join zesp;
```

32. Zdefiniuj klucz podstawowy na atrybucie ID_ZESP relacji ZESP. Wykonaj ponownie oba zapytania z zadania 31.

```
alter table zesp add constraint zesp_pk primary key(id_zesp);
```

33. Utwórz indeks typu B-drzewo o nazwie PRAC_ID_ZESP_IDX na atrybucie ID_ZESP relacji PRAC. Wykonaj ponownie oba zapytania z zadania 31.

```
create index prac_id_zesp_idx on prac(id_zesp);
```

34. Utwórz relacje PRACOWNICY i ETATY, wykonując skrypt **`pldemobld.sql`** z katalogi **`/ora1/oracle`**. Następnie wykonaj poniższe zapytanie. Czemu optymalizator użył algorytmu sort merge?

```
select * from pracownicy join etaty on placa_pod between placa_min and
placa_max;
```

35. Utwórz bitmapowy indeks o nazwie JOIN_IND na wyniku operacji połączenia naturalnego relacji PRAC i ZESP. Poindeksowanym atrybutem ma być atrybut NAZWA relacji ZESP.

```
create bitmap index join_ind on prac(nazwa)
from prac p, zesp z
where p.id_zesp=z.id_zesp;
```

36. Sprawdź plan wykonania poniższego zapytania. Czy optymalizator użył utworzonego w p. 35. indeksu?

```
select count(*) from prac join zesp using(id_zesp)
where nazwa='Algorytmy';
```

Statystyki, histogramy

37. Usuń statystyki dla tabeli PRAC. Następnie dokonaj oszacowania statystyk dla tabeli PRAC na podstawie próbki 10%. Wyłącz tryb wyświetlania planu zapytania i sprawdź informacje o statystykach dla tabeli PRAC (podobnie jak w zadaniu 18.).

```
exec dbms_stats.delete_table_stats(ownname=>'OPT_USER',tabname=>'PRAC');
exec dbms_stats.gather_table_stats(ownname=>'OPT_USER',tabname=>'PRAC',
estimate_percent=>10);
```

38. Poznaj rozkład wartości atrybutu PLACA_DOD w tabeli PRAC.

```
select placa_dod, count(*)
from prac group by placa_dod;
```

39. Usuń wszystkie statystyki dla tabeli PRAC (również statystyki dla atrybutów i indeksów). Utwórz indeks typu B-drzewo o nazwie PRAC_PLACA_DOD_IDX na atrybucie PLACA_DOD tabeli PRAC. Włącz tryb wyświetlania plany zapytania.

```
exec dbms_stats.delete_table_stats(ownname=>'OPT_USER',tabname=>'PRAC');

create index prac_placa_dod_idx on prac(placa_dod);
```

40. Wyświetl plany poniższych zapytań

```
select /*+ dynamic_sampling(0)*/ * from prac where placa_dod = 100;
select /*+ dynamic_sampling(0)*/* from prac where placa_dod = 999;
```

Co zaobserwowałeś/aś?

Uwaga! Zamieszczona w zadaniu wskazówka wyłącza automatyczne próbkowanie statystyk przed wykonaniem polecenia w przypadku braku zebranych statystyk.

41. Utwórz histogram dla atrybutu PLACA_DOD tabeli PRAC. Powtórz zapytania z zadania 40.

```
exec dbms_stats.gather_table_stats(ownname=>'OPT_USER',tabname=>'PRAC',
method_opt=>'FOR ALL COLUMNS placa_dod SIZE AUTO');
```

Wskazówki

42. Usuń indeks bitmapowy PRAC_PLEC_BMP_IDX, założony na atrybucie PLEC tabeli PRAC. Następnie utwórz na tym samym atrybucie indeks B-drzewo o nazwie PRAC_PLEC_IDX.

```
drop index PRAC_PLEC_BMP_IDX;
create index prac_plec_idx on prac(plec);
```

43. Dostęp do danych. Sprawdź plan poniższego zapytania. Następnie dodaj do zapytania taką wskazówkę, aby optymalizator kosztowy użył utworzonego w zadaniu 42. indeksu.

```
select count(*) from prac where id_prac < 100 and plec = 'K';
```

```
select /*+INDEX (prac prac_plec_idx)*/ count(*)
from prac where id_prac < 100 and plec = 'K';
```

44. Dostęp do danych. Wykonaj ponownie zapytanie z zadania 43, tym razem dodaj wskazówkę, która spowoduję, że optymalizator nie użyje żadnego indeksu przy dostępie do tabeli PRAC.

```
select /*+NO_INDEX (prac)*/ count(*)
from prac where id_prac < 100 and plec = 'K';
```

45. Kolejność łączenia tabel. Sprawdź plan wykonania poniższego zapytania:

```
select * from pracownicy join etaty on placa_pod between placa_min and
placa_max natural join zespoly;
```

Następnie dodaj do zapytania wskazówkę ORDERED. Co zaobserwowałeś/aś? Zmień kolejność relacji w klauzuli from i ponownie wykonaj zapytanie.

46. Wykonaj ponownie zadanie 45, tym razem do zmiany kolejności łączenia tabel użyj wskazówki LEADING.

47. Porównaj koszty wykonania połączenia naturalnego tabel PRAC i ZESP za pomocą każdego algorytmu łączenia tabel i dla każdej kolejności łączonych tabel (6 kombinacji). Określ plan charakteryzujący się najniższym kosztem.

*SQL*Trace*

48. Ustaw parametr TIMED_STATISTICS = true w pliku inicjalizacyjnym bazy danych.

49. Wyłącz wyjaśnianie planu zapytania.

```
set autotrace off
```

50. Jako użytkownik SYS sprawdź wartość zmiennej USER_DUMP_DEST, wskazującej katalog, w którym zostanie umieszczony plik śladu.

```
show parameter user_dump_dest
```

51. Uruchom narzędzie *SQL*Trace*, wykonaj kilka zapytań, następnie wyłącz *SQL*Trace*.

```
alter session set sql_trace=true;
select count(*) from prac group by id_zesp;
update prac set placa=placa+34
where plec='K' and czy_etat='T' and id_prac < 100;
alter session set sql_trace=false;
```

52. Użyj programu *TKProf* do sformatowania wyniku, który znajduje się w pliku śladu

```
tkprof <plik_sladu> plik.txt aggregate=yes sys=no
explain=opt_user/opt_user
```

Obejrzyj wynik i usuń pliki

53. Jako użytkownik SYS odczytaj z perspektywy v$session dane o sesji użytkownika OPT_USER.

```
select sid, serial# from v$session
where username = 'OPT_USER';
```

Następnie uruchom niewidocznie dla użytkownika OPT_USER śledzenie poleceniem (w miejsce sid i serial# wpisz wartości uzyskane w poprzednim zapytaniu):

```
exec dbms_system.set_sql_trace_in_session(<sid>,<serial#>,true);
```

54. Jako OPT_USER wykonaj kilka zapytań:

```
select count(*) from prac;
select max(placa) from prac;
delete from prac_zesp_iot;
```

55. Jako SYS zakończ śledzenie sesji użytkownika OPT_USER poleceniem:

```
exec dbms_system.set_sql_trace_in_session(<sid>,<serial#>,false);
```

56. Użyj programu TKProf do sformatowania wyniku, który znajduje się w pliku:

```
tkprof <plik_sladu> plik.txt aggregate=yes sys=no
explain=opt_user/opt_user
```

Obejrzyj wynik i usuń pliki